

LAPORAN PRAKTIKUM BIG DATA
MINGGU 2: PENGUMPULAN DAN PRAPEMROSESAN DATA (DATA
ACQUISITION & PREPROCESSING)



OLEH:
ALFARIS AULIA RAHMAN
2411533006

DOSEN PENGAMPU:
AFDHAL DINILHAK, S.KOM, M.KOM.

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

A. Pendahuluan

Data transaksional dari sistem marketplace sering mengandung nilai kosong, pencatatan ganda, tipe data yang tidak sesuai, serta format teks dan tanggal yang tidak konsisten. Jika langsung digunakan, masalah tersebut dapat menghasilkan jumlah transaksi yang berlebihan, statistik yang bias, dan kesimpulan yang tidak dapat dipercaya.

Praktikum ini mensimulasikan proses pengumpulan data transaksi menggunakan dataset sintetis dengan random seed tetap. Dataset mentah terdiri atas 515 baris, yaitu 500 transaksi dasar dan 15 baris duplikat. Tahap preprocessing mencakup penanganan missing value, penghapusan duplicate, standardisasi kategori, pembersihan harga, serta penyamaan tanggal ke format YYYY-MM-DD.

Laporan ini menjelaskan landasan teori, langkah pengerjaan K-1 sampai K-6, analisis hasil, jawaban studi kasus, hasil latihan, dan jawaban pertanyaan evaluasi. Hasil akhir berupa transaksi_bersih.csv yang berisi 490 baris dan siap digunakan pada tahap penyimpanan berikutnya.

B. Tujuan

1. Membuat dataset transaksi marketplace sintetis yang dapat direproduksi.
2. Mendeteksi dan menangani missing value sesuai fungsi setiap kolom.
3. Mendeteksi dan menghapus transaksi duplicate secara aman.
4. Mengoreksi tipe data serta menstandarisasi format teks, harga, dan tanggal.
5. Mengekspor dataset bersih dalam format CSV.
6. Menjelaskan hubungan preprocessing dengan dimensi Veracity pada konsep 5V Big Data.

C. Dasar Teori dan Dataset

1. Data Acquisition dan Preprocessing

Data acquisition adalah proses memperoleh data dari suatu sumber. Pada praktikum ini, acquisition disimulasikan melalui pembentukan data transaksi menggunakan Python, NumPy, random, dan Faker. Preprocessing adalah serangkaian tindakan untuk meningkatkan konsistensi dan keterpercayaan data sebelum data dianalisis atau disimpan secara terstruktur.

2. Missing Value

Missing value merupakan kondisi ketika suatu atribut tidak memiliki nilai. Strategi penanganannya harus mengikuti makna kolom. `customer_name` dan `payment_method` bersifat wajib sehingga barisnya dibuang, `shipping_city` masih dapat ditandai sebagai Tidak Diketahui, sedangkan `rating` dibiarkan kosong karena memang bersifat opsional.

3. Duplicate

Duplicate adalah pencatatan data yang sama lebih dari sekali. Jika tidak dihapus, transaksi ganda dapat meningkatkan jumlah transaksi dan nilai agregasi secara keliru. Fungsi `duplicated()` digunakan untuk mendeteksi, sedangkan `drop_duplicates()` digunakan untuk menghapusnya.

4. Standardisasi Tipe dan Format

Standardisasi membuat nilai dalam satu kolom mengikuti aturan yang seragam. Teks dibersihkan dari spasi berlebih dan disamakan kapitalisasinya. Harga diubah dari teks bercampur simbol menjadi float. Tanggal diproses menggunakan format eksplisit agar format ISO tidak salah ditafsirkan akibat penggunaan `dayfirst=True`.

5. Veracity

Veracity menunjukkan tingkat akurasi, konsistensi, dan keterpercayaan data. Proses cleaning pada praktikum ini meningkatkan Veracity karena mengurangi pencatatan ganda, menangani informasi wajib yang hilang, serta menyatukan format data.

6. Struktur Dataset

Kolom	Tipe yang Diharapkan	Keterangan
<code>transaction_id</code>	teks	ID unik transaksi
<code>customer_name</code>	teks	Nama pembeli
<code>product_name</code>	teks	Nama produk
<code>category</code>	kategorikal	Kategori produk
<code>price</code>	float	Harga satuan
<code>quantity</code>	integer	Jumlah barang
<code>payment_method</code>	kategorikal	Metode pembayaran
<code>transaction_date</code>	tanggal	Tanggal transaksi
<code>shipping_city</code>	teks	Kota tujuan
<code>rating</code>	numerik, opsional	Rating 1-5

D. Langkah Pengerjaan Praktikum

1. Import Library dan Inisialisasi

```
[1] ✓ 13s !pip install faker -q
2.1/2.1 MB 26.3 MB/s eta 0:00:00

[2] ✓ 0s import numpy as np
import pandas as pd
from faker import Faker
import random
```

Library NumPy, pandas, Faker, dan random diimpor. NumPy dan random digunakan untuk pemilihan nilai, Faker menghasilkan nama, kota, dan tanggal sintetis, sedangkan pandas menangani data tabular.

2. Membuat Dataset Sintetis

Random seed ditetapkan menjadi 42 agar proses dapat direproduksi. Sebanyak 500 transaksi dibuat, kemudian missing value disuntikkan secara terkendali dan 15 baris diduplikasi. Dataset diacak dan disimpan sebagai transaksi_mentah.csv.

```
[3] ✓ 0s SEED = 42
np.random.seed(SEED)
random.seed(SEED)
fake = Faker("id_ID")
Faker.seed(SEED)

N = 500
kategori_produk = ["Elektronik", "Fashion", "Kesehatan", "Rumah Tangga", "Olahraga", "Buku"]
metode_bayar = ["Transfer Bank", "E-Wallet", "COD", "Kartu Kredit"]

rows = []
for i in range(1, N + 1):
    trx_id = f"TRX{i:05d}"
    nama_pelanggan = fake.name()
    produk = fake.word().capitalize() + " " + random.choice(["Pro", "Lite", "Max", "Basic", ""])
    kategori = random.choice(kategori_produk)
    harga_dasar = random.choice([15000, 25000, 50000, 75000, 120000, 250000, 500000, 1200000])
    qty = random.randint(1, 5)

    # Variasi format harga: angka polos, ada "Rp", ada desimal ".", ada spasi
    harga_variants = [
        str(harga_dasar),
        f"Rp{harga_dasar:,}".replace(",", "."),
        f"{harga_dasar}.0",
        f" {harga_dasar} ",
    ]
    harga = random.choice(harga_variants)

    # Variasi format tanggal: ISO, DD/MM/YYYY, DD-MM-YYYY
    tgl = fake.date_between(start_date="-90d", end_date="today")
    tgl_variants = [tgl.strftime("%Y-%m-%d"), tgl.strftime("%d/%m/%Y"), tgl.strftime("%d-%m-%Y")]
    tanggal = random.choice(tgl_variants)

    metode = random.choice(metode_bayar)
    if random.random() < 0.3:
        metode = metode.lower()
    if random.random() < 0.2:
        kategori = kategori.upper() + " "

    kota = fake.city()
    rating = random.choice([1, 2, 3, 4, 5, None, None]) # rating opsional

    rows.append({
        "transaction_id": trx_id, "customer_name": nama_pelanggan, "product_name":
        produk.strip(),
        "category": kategori, "price": harga, "quantity": qty, "payment_method": metode,
        "transaction_date": tanggal, "shipping_city": kota, "rating": rating,
    })

df = pd.DataFrame(rows)
```

```
# Suntikkan missing value pada beberapa kolom
for col, frac in [("customer_name", 0.02), ("shipping_city", 0.03), ("payment_method",
0.015)]:
    idx = df.sample(frac=frac, random_state=SEED).index
    df.loc[idx, col] = np.nan

# Duplikasi 15 baris (mensimulasikan transaksi yang tercatat dua kali)
dup_rows = df.sample(n=15, random_state=SEED)
df = pd.concat([df, dup_rows], ignore_index=True)
df = df.sample(frac=1, random_state=SEED).reset_index(drop=True)

df.to_csv("transaksi_mentah.csv", index=False)
print("Jumlah baris:", len(df))
```

Jumlah baris: 515

3. Deteksi dan Penanganan Missing Value

Pemeriksaan menggunakan `isnull().sum()` menunjukkan missing value pada `customer_name`, `payment_method`, `shipping_city`, dan `rating`.

```
[4]: print(df.isnull().sum())
transaction_id      0
customer_name      20
product_name        0
category            0
price               0
quantity            0
payment_method      16
transaction_date     0
shipping_city       30
rating              166
dtype: int64
```

Baris tanpa `customer_name` atau `payment_method` dibuang karena kedua atribut tersebut wajib. `shipping_city` diisi dengan Tidak Diketahui, sedangkan `rating` tetap dibiarkan kosong.

```
[5]: df = df.dropna(subset=["customer_name", "payment_method"])
df["shipping_city"] = df["shipping_city"].fillna("Tidak Diketahui")
```

4. Deteksi dan Penanganan Duplicate

```
[6]: print("Baris duplicate (semua kolom sama):", df.duplicated().sum())
print("transaction_id duplicate:", df["transaction_id"].duplicated().sum())

df = df.drop_duplicates()
print("Jumlah baris setelah drop_duplicates():", len(df))

Baris duplicate (semua kolom sama): 5
transaction_id duplicate: 5
Jumlah baris setelah drop_duplicates(): 490
```

Setelah penanganan missing value, duplicate diperiksa berdasarkan seluruh kolom dan `transaction_id`. Terdapat 5 duplicate yang masih tersisa dan seluruhnya dihapus sehingga jumlah data menjadi 490 baris.

5. Koreksi Tipe Data dan Standardisasi Format

```

a. Standardisasi teks kategorikal (category, payment_method, shipping_city)

[7]
✓ Os
for col in ["category", "payment_method", "shipping_city"]:
    df[col] = df[col].astype("string").str.strip().str.title()

# "Cod" adalah singkatan; kembalikan ke huruf kapital penuh setelah Title Case
df["payment_method"] = df["payment_method"].replace({"Cod": "COD"})

b. Koreksi tipe data pada kolom price (dari teks bercampur simbol menjadi numerik)

[8]
✓ Os
def bersihkan_harga(x):
    if pd.isna(x):
        return np.nan
    x = str(x).strip().replace("Rp", "").replace(".", "").replace(",", ".")
    try:
        return float(x)
    except ValueError:
        return np.nan

df["price"] = df["price"].apply(bersihkan_harga)

c. Standardisasi format tanggal ke YYYY-MM-DD

[9]
✓ Os
def parse_tanggal(x):
    for fmt in ("%Y-%m-%d", "%d/%m/%Y", "%d-%m-%Y"):
        try:
            return pd.to_datetime(x, format=fmt)
        except ValueError:
            continue
    return pd.NaT

df["transaction_date"] = df["transaction_date"].apply(parse_tanggal).dt.strftime("%Y-%m-%d")

d. Finalisasi tipe data

[10]
✓ Os
df["quantity"] = df["quantity"].astype(int)
df["price"] = df["price"].astype(float)

```

Kolom category, payment_method, dan shipping_city diubah ke pandas string, dibersihkan dari spasi, dan diseragamkan dengan title case. Singkatan COD dikembalikan ke huruf kapital. Price dibersihkan dari Rp dan pemisah ribuan, sedangkan tanggal diparse satu per satu dengan tiga format eksplisit.

6. Ekspor Dataset Bersih

```

[11]
✓ Os
df.to_csv("transaksi_bersih.csv", index=False)
print("Dataset bersih tersimpan:", len(df), "baris")

Dataset bersih tersimpan: 490 baris

```

Dataset akhir diekspor dengan nama transaksi_bersih.csv tanpa kolom indeks. File ini berisi 490 baris dan menjadi keluaran utama praktikum.

E. Analisis Hasil

Dataset mentah memiliki 515 baris dan dataset bersih memiliki 490 baris. Dengan demikian, 25 baris atau sekitar 4,85 persen tidak dipertahankan dalam hasil akhir. Jumlah tersebut bukan berarti seluruh 25 baris merupakan masalah yang berbeda, karena baris duplikat dapat sekaligus mengandung missing value wajib.

Pada data mentah ditemukan 20 missing customer_name, 16 missing payment_method, 30 missing shipping_city, dan 166 missing rating. Angka customer_name dan payment_method mencakup salinan dari baris yang diduplikasi. Ketika dropna() dijalankan, 20 baris dibuang.

Dari 15 baris duplikat yang disuntikkan, 10 pasang berkaitan dengan baris yang kehilangan informasi wajib sehingga ikut terhapus pada tahap dropna(). Oleh sebab itu, ketika duplicate diperiksa sesudahnya, hanya 5 duplicate yang masih tersisa. Lima baris tersebut kemudian dihapus dan jumlah akhir menjadi 490.

Perbedaan tersebut menjelaskan mengapa hasil eksekusi aktual menampilkan 5 duplicate pada tahap K-4, bukan 15. Jumlah akhir tetap sesuai target, yaitu 490 baris. Setelah standardisasi, category memiliki enam nilai unik, payment_method memiliki empat nilai unik, price bertipe float, dan transaction_date mengikuti pola YYYY-MM-DD.

F. Studi Kasus

Platform marketplace kita mendeteksi kejanggalan pada laporan penjualan bulanan: total transaksi yang dilaporkan tim IT (515) tidak sama dengan total yang dipakai tim Finance (490). Sebagai calon data engineer, jelaskan kepada tim Finance:

1. Mengapa kedua angka tersebut bisa berbeda, dikaitkan dengan proses yang baru saja Anda lakukan.
2. Apakah 490 baris “lebih benar” dibanding 515 baris? Jelaskan dengan mengaitkan ke konsep Veracity.
3. Bagaimana Anda akan menjelaskan keputusan membiarkan kolom rating tetap memiliki missing value kepada tim Finance yang ingin tahu “rating rata-rata semua transaksi”?

Jawab:

1. Tim IT menghitung seluruh 515 baris yang diterima pada data mentah, termasuk baris duplicate dan transaksi yang kehilangan informasi wajib. Tim Finance menggunakan 490 baris setelah preprocessing. Selisih tersebut muncul karena proses pembersihan menghapus baris yang tidak layak dipakai dan mencegah transaksi yang sama dihitung berulang kali.
2. Untuk kebutuhan analisis Finance, 490 baris lebih dapat dipercaya karena tidak memuat pencatatan ganda dan transaksi yang kehilangan customer_name atau payment_method. Kondisi ini meningkatkan Veracity. Namun, angka 515 tetap benar jika konteksnya adalah jumlah baris mentah yang diterima sebelum pembersihan.
3. Rating tidak diisi dengan angka tebakan karena pemberian rating bersifat opsional. Imputasi tanpa dasar dapat mengubah distribusi dan rata-rata rating. Rata-rata sebaiknya

dihitung hanya dari transaksi yang memiliki rating, disertai informasi jumlah rating tersedia dan jumlah nilai kosong.

G. Latihan

1. Latihan 1: Ubah SEED menjadi 7 dan jalankan ulang seluruh pipeline. Bandingkan jumlah baris transaksi_mentah.csv dan transaksi_bersih.csv dengan hasil SEED = 42. Apakah jumlahnya sama? Jelaskan mengapa.
2. Latihan 2: Tambahkan kolom is_valid_price bernilai True jika price > 0. Gunakan untuk memeriksa apakah ada harga tidak valid.
3. Latihan 3: Hitung jumlah transaksi per category menggunakan value_counts() pada dataset yang sudah bersih.

Jawab:

1. Seluruh pipeline dijalankan ulang dengan SEED 7. Dataset mentah tetap berisi 515 baris dan dataset bersih tetap berisi 490 baris. Seed mengubah data yang dihasilkan dan baris yang terpilih, tetapi tidak mengubah jumlah transaksi dasar, jumlah duplicate yang disuntikkan, maupun proporsi missing value.

```
[12] SEED = 7
      np.random.seed(SEED)
      random.seed(SEED)
      fake = Faker("id_ID")
      Faker.seed(SEED)

      N = 500
      kategori_produk = ["Elektronik", "Fashion", "Kesehatan", "Rumah Tangga", "Olahraga",
                        "Buku"]

[14] df.to_csv("transaksi_bersih_7.csv", index=False)
      print("Dataset bersih tersimpan:", len(df), "baris")

      Dataset bersih tersimpan: 490 baris
```

2. Kolom is_valid_price bernilai True jika price lebih besar dari nol. Seluruh 490 transaksi memiliki harga valid sehingga tidak ditemukan harga bernilai nol atau negatif.

```
[21] df["is_valid_price"] = df["price"] > 0

      print(df["is_valid_price"].value_counts())
      print("Jumlah harga tidak valid:", (~df["is_valid_price"]).sum())

      is_valid_price
      True    490
      Name: count, dtype: int64
      Jumlah harga tidak valid: 0
```

3. Kategori dengan jumlah transaksi tertinggi adalah Rumah Tangga sebanyak 91 transaksi, sedangkan jumlah terendah terdapat pada Olahraga sebanyak 74 transaksi.

```
[22] print(df["category"].value_counts())

      category
      Rumah Tangga    91
      Kesehatan       86
      Buku            81
      Fashion         80
      Elektronik       78
      Olahraga        74
      Name: count, dtype: int64
```

H. Pertanyaan Evaluasi

1. Apa perbedaan antara missing value yang ditangani dengan `dropna()` dan yang ditangani dengan `fillna()`? Kapan masing-masing sebaiknya dipakai?
2. Mengapa `drop_duplicates()` sebaiknya dijalankan pada tahap awal pipeline, bukan di akhir?
3. Jelaskan risiko dari kode `pd.to_datetime(df["transaction_date"], format="mixed", dayfirst=True)` pada data yang formatnya tercampur ISO dan non-ISO.
4. Mengapa kolom `rating` tidak diisi (di-impute) meskipun mengandung banyak missing value, sedangkan kolom `shipping_city` diisi dengan “Tidak Diketahui”?
5. Bagaimana keterkaitan antara praktikum ini dengan dimensi Veracity pada 5V?

Jawab:

1. `dropna()` membuang baris yang memiliki missing value dan digunakan ketika nilai yang hilang merupakan informasi wajib sehingga baris tidak dapat dipakai secara valid. `fillna()` mempertahankan baris dengan mengganti nilai kosong memakai penanda yang masuk akal. Pada praktikum ini, `customer_name` dan `payment_method` ditangani dengan `dropna()`, sedangkan `shipping_city` diisi dengan Tidak Diketahui.
2. Duplicate sebaiknya ditangani sebelum analisis agar transaksi yang sama tidak memengaruhi jumlah, total, atau rata-rata. Penghapusan lebih awal juga mengurangi data yang harus diproses pada tahap berikutnya. Pada pipeline modul, pemeriksaan duplicate dilakukan setelah penanganan missing value tetapi tetap sebelum standarisasi dan analisis.
3. Kombinasi format `mixed` dan `dayfirst=True` dapat salah menafsirkan tanggal ISO yang sebenarnya tidak ambigu. Nilai `2026-07-11` dapat dibaca sebagai 7 November 2026. Pendekatan yang lebih aman adalah mencoba format `%Y-%m-%d`, `%d/%m/%Y`, dan `%d-%m-%Y` secara eksplisit untuk setiap nilai.
4. `Rating` dibiarkan kosong karena bersifat opsional dan pengisian dengan angka tebakan dapat menghasilkan rata-rata yang bias. `Shipping_city` diisi dengan Tidak Diketahui karena barisnya masih bermanfaat untuk analisis lain dan label tersebut tidak mengarang lokasi tertentu.
5. Praktikum ini meningkatkan Veracity melalui deteksi dan penanganan missing value, penghapusan duplicate, koreksi tipe data, serta standarisasi format. Data bersih

menjadi lebih konsisten dan dapat dipercaya sebagai dasar penyimpanan dan analisis berikutnya.

I. Kesimpulan

Praktikum berhasil mensimulasikan acquisition data transaksional dan menjalankan preprocessing secara lengkap. Dataset mentah berisi 515 baris dengan missing value, duplicate, serta format yang tidak konsisten. Setelah pembersihan, diperoleh transaksi_bersih.csv berisi 490 baris. Seluruh harga valid, format tanggal telah seragam, dan kategori serta metode pembayaran telah distandardisasi.

Proses ini menunjukkan bahwa kualitas data tidak cukup dinilai dari volumenya. Keputusan penanganan harus mempertimbangkan makna setiap kolom agar peningkatan Veracity tidak menciptakan bias baru. Dataset akhir telah siap digunakan pada tahap penyimpanan berikutnya.

Github : [BigData26_A_2411533006_AlfarisAuliaRahman](#)

Google Drive : [BigData](#)